Win probability in the lab and in the app

One curve, two clamps, and the move labels they can change

Chess Coach research lab

Abstract

The lab and the coaching app turn an engine evaluation into a win probability with the same curve, the logistic Lichess fits on its games, with a slope of 0.00368208 per centipawn. They clamp it differently: the lab clamps the evaluation at ±10 pawns and reads every mate as that clamp, while the app clamps at ±100 pawns and reads mate in n as $\pm(10000 - 10n)$ centipawns. We evaluated both at 2,461 evaluations, the app's through its own code at commit `d8b7712`. Within ±10 pawns the two agree at every point. Past it the lab holds every evaluation, mates included, at 97.54% for the side ahead, while the app goes on to 100%: a gap of up to 2.46 points. The lab's move labels (a loss of more than 10 or 20 points of the mover's win probability) can therefore differ from the same labels read on the app's curve, but only for a move that starts more than 10 pawns ahead or ends more than 10 pawns behind, and only by one step, the lab's being the milder.

1 Why the two must agree

The lab trains its models on labels read from win probability. Its data pipeline marks a move as a blunder when it costs the mover more than 20 points of win probability (the blunder hazard model's target, and the early traps the repertoire optimizer counts), and the graded-opponent scorer keeps a move as a candidate when it costs more than 10. A model that wins in the lab is ported to the app, and the app computes its own verdicts from its own win probability. If the two curves differ, a label the lab trained on means something else once the model runs in the app.

The lab never re-implements an app instrument to compare against it. This check calls the app's own function (`winPercent` in `src/core/score`) at the commit the lab pins, through the lab's bridge, and compares it with the lab's (`win_percent` in `chess_coach_models.winprob`).

2 One curve, two clamps

Both sides compute White's win probability from an evaluation of x centipawns from White's side as

$$W(x) \;=\; 50 + 50\left(\frac{2}{1 + e^{-kx}} - 1\right), \qquad k = 0.00368208,$$

and differ only in what they feed it.

- **The lab** clamps x to ±1000 before the curve, as Lichess does. A mate from a game's annotations, or from the engine, becomes ±10000 (or within a few centipawns of it) and is then clamped like any other evaluation, so every mate reads 97.54% for the side delivering it and 2.46% for the side receiving it.

- **The app** clamps x to ± 10000 and reads mate in n as $\pm(10000 - 10n)$, so a mate reads as won, a little less won the further off it is. Its code notes that the wider clamp is the one every number it has stored was computed with. A finished game or a tablebase result reads exactly 100, 50 or 0; the lab has no such reading, and this check leaves it out.

3 Method

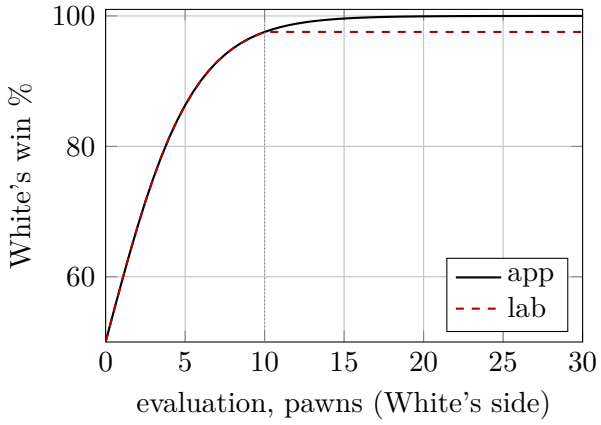
Both functions were evaluated at every evaluation from -120 to $+120$ pawns in steps of 0.1 pawns and at mate in 1 to 30 for either side: 2,461 points, 201 of them within the lab's clamp. The lab's side runs in Python, the app's in Node under the app's own toolchain at `d8b7712` (`parity.py` sends the evaluations to `app.mts` on standard input and reads White's win probability back). The check fails if any point within ± 10 pawns differs by more than 10^{-9} points, the room left for the two languages' exponentials to differ in the last bit. Past the clamp it reports, for each range, the largest gap and where it falls.

4 Results

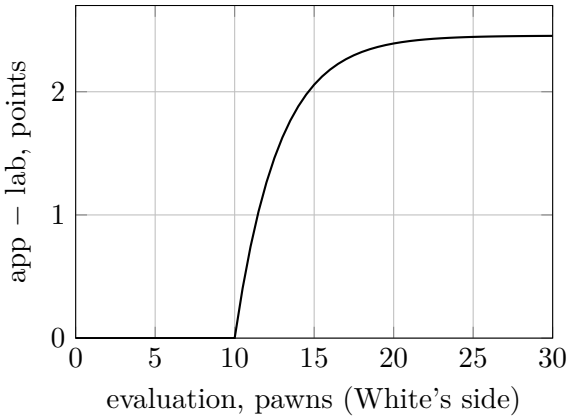
Within ± 10 pawns the two curves agree at all 201 points (the largest difference rounds to 0 at six decimals). Past the clamp they part as designed (Table 1, Figure 1): the lab's curve is flat at 97.54% while the app's keeps rising, and from 25.5 pawns on the app reads the side ahead as won to within 0.01 points. Every mate shows the full gap, 2.46 points, because the lab reads it at the clamp and the app at 100% less a sliver.

range	points	largest gap	at	lab	app
within ± 1000 cp	201	0.0000	—	37.3907	37.3907
± 1000 to ± 10000 cp	1,800	2.4553	+9980 cp	97.5447	100.0000
beyond ± 10000 cp	400	2.4553	+10010 cp	97.5447	100.0000
mate in 1 to 30	60	2.4553	#+1	97.5447	100.0000

Table 1: White's win probability in the lab and in the app, in percentage points: the largest gap in each range, the first evaluation where it occurs, and the two readings there. Within the lab's clamp the gap is zero at every point.



(a) The two curves. The dotted line is the lab's clamp.



(b) The gap. It is zero up to the clamp and approaches 2.46 points past it.

Figure 1: White's win probability for White's evaluations from level to 30 pawns ahead, every half pawn, both columns written by `parity.py`. Black's side is the mirror image.

5 Which labels can change

A move's label depends on its loss, the mover's win probability before the move less the one after. Read from the mover's side, the two curves give the same loss when both evaluations lie within the clamp, and the same label when both lie past it on the same side (a loss of at most 2.46 points either way, too small for any label) or on opposite sides (a change of more than 95.09 points on either curve: a blunder on both, or a gain on both). A label can differ only when one evaluation lies past the clamp and the other within it, and of those only a move that starts more than 10 pawns ahead, or ends more than 10 pawns behind, has a positive loss to label. There the lab's loss is smaller than the app's by the gap between the two readings of the evaluation past the clamp, at most 2.46 points, so the lab's label is one step milder whenever the app's loss exceeds a threshold by no more than that gap.

The claim, checked. Over every move from one evaluation to another on a grid of 481 evaluations (every half pawn from −120 to +120; 231,361 pairs, both curves as `parity.py` wrote them), 872 get different labels: 436 blunders on the app's curve are candidates in the lab, and 436 candidates on the app's curve are unlabelled in the lab. Every one starts more than 10 pawns ahead for the mover or ends more than 10 pawns behind, in every one the lab's label is one step milder, and no window is wider than 2.46 points. `generate.py` checks all three and refuses to write this paper if any fails. The counts say how many grid pairs are affected, not how many moves in real games are.

An example. A player who is +15.00 pawns up plays a move that leaves them +3.50 pawns up, the smallest such drop the app's curve calls a blunder. The app reads the position before at 99.60% and after at 78.39%, a loss of 21.21 points: a blunder. The lab reads the position before at 97.54% and after at the same 78.39%, a loss of 19.15 points: a candidate, not a blunder. In a won position, the lab forgives a little more.

6 What a port has to decide

Which clamp a ported model uses is decided in its port, not here. A model whose labels or features were computed with the lab's clamp either keeps that clamp inside the port and says so, or is retrained on labels read from the app's curve. This check is how either choice stays honest: it runs again whenever the lab moves its pin to a new app commit (`make parity`), and fails if the curves ever differ within the clamp.

7 Reproduction

`parity.py` writes the report (`results/parity_winprob.json` and `.md`) and the curve every half pawn (`results/parity_winprob_curve.csv`) from the app at `d8b7712`; `paper/generate.py` reads those files, the lab's thresholds and the lab's own curve, and writes every number this paper prints. The app's repository is private, so the check runs where the bridge can read it, and its results are committed with the commit they measured.

8 Limitations

- The grid is uniform in evaluation. How often real moves fall in the window where a label differs, on Lichess games at the levels the lab trains on, is not measured here.
- Only evaluations and mates are compared. A finished game or a tablebase result, which the app reads exactly, has no counterpart in the lab.